

kd-trees

Nearest neighbour by scanning

Algorithm: NearestNeighbour (D, x_q)

input : Data set D , query point x_q

output: Nearest neighbour nn , $\forall x \in D. d(nn, q) \leq d(x, q)$

```
1 nndist  $\leftarrow \infty$ 
2 nn  $\leftarrow -$ 
3 foreach  $x$  in  $D$  do
4     dist  $\leftarrow d(x, x_q)$ 
5     if dist < nndist then
6         nndist  $\leftarrow$  dist
7         nn  $\leftarrow x$ 
8     end
9 end
10 return nn
```

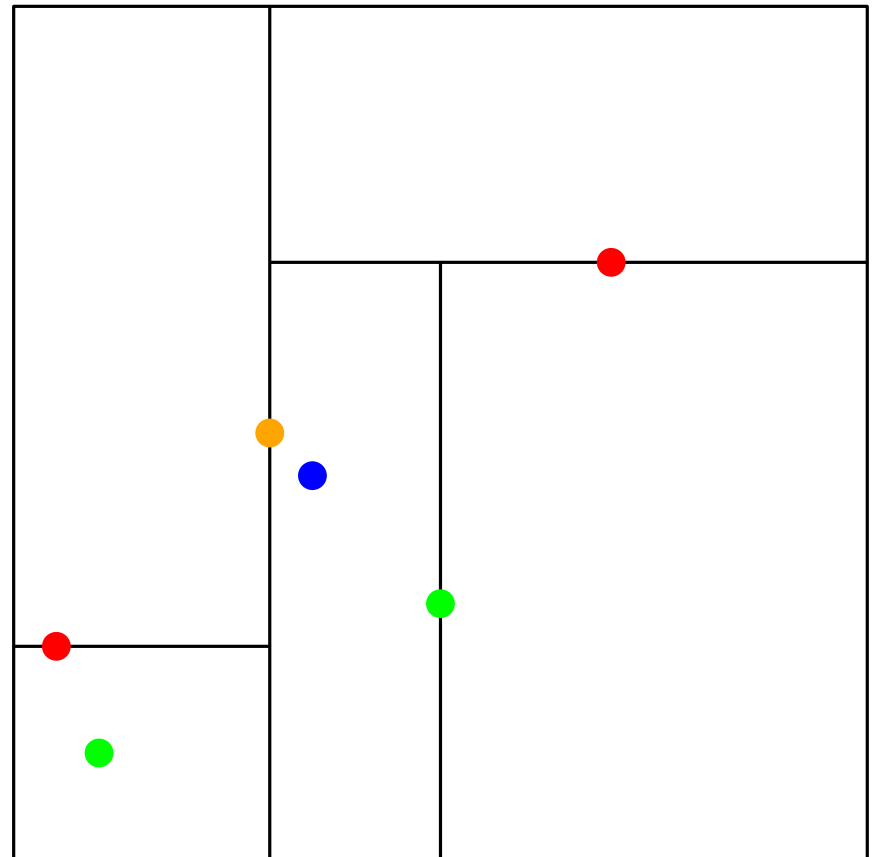
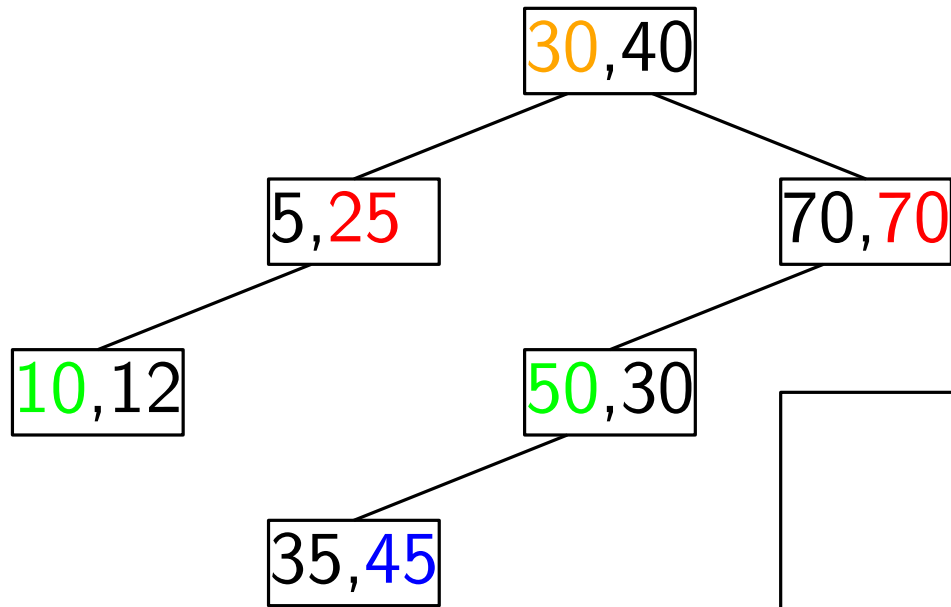
kd-tree example

cd = x

cd = y

cd = x

cd = y



kd-tree insert

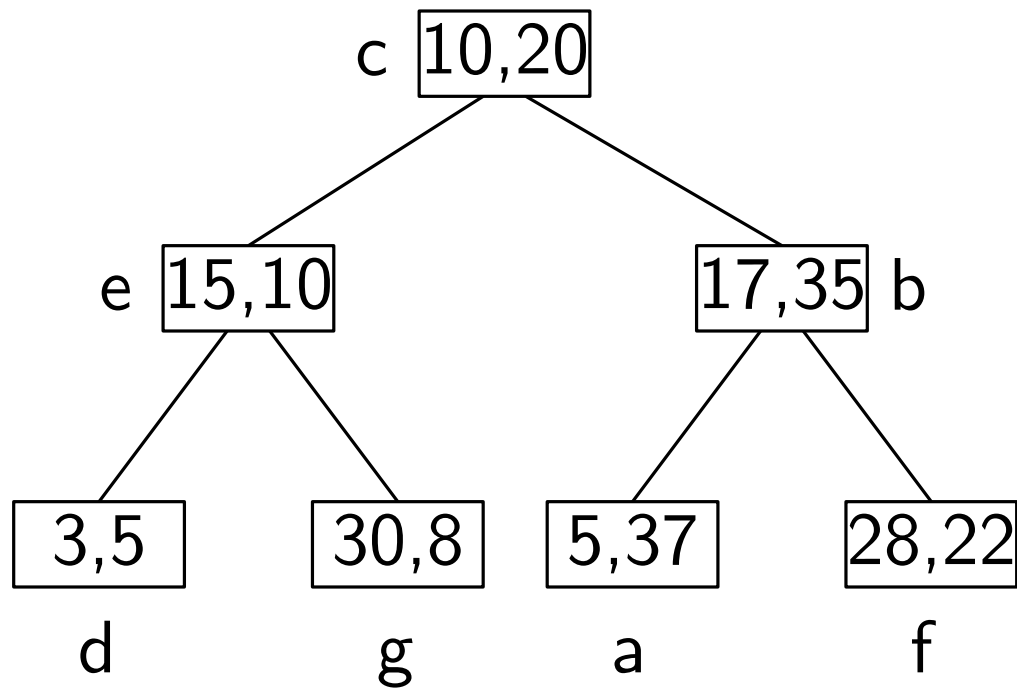
Algorithm: kdt-insert (t , q , cd)

input : kd-tree t , query point q , cutting dimension cd

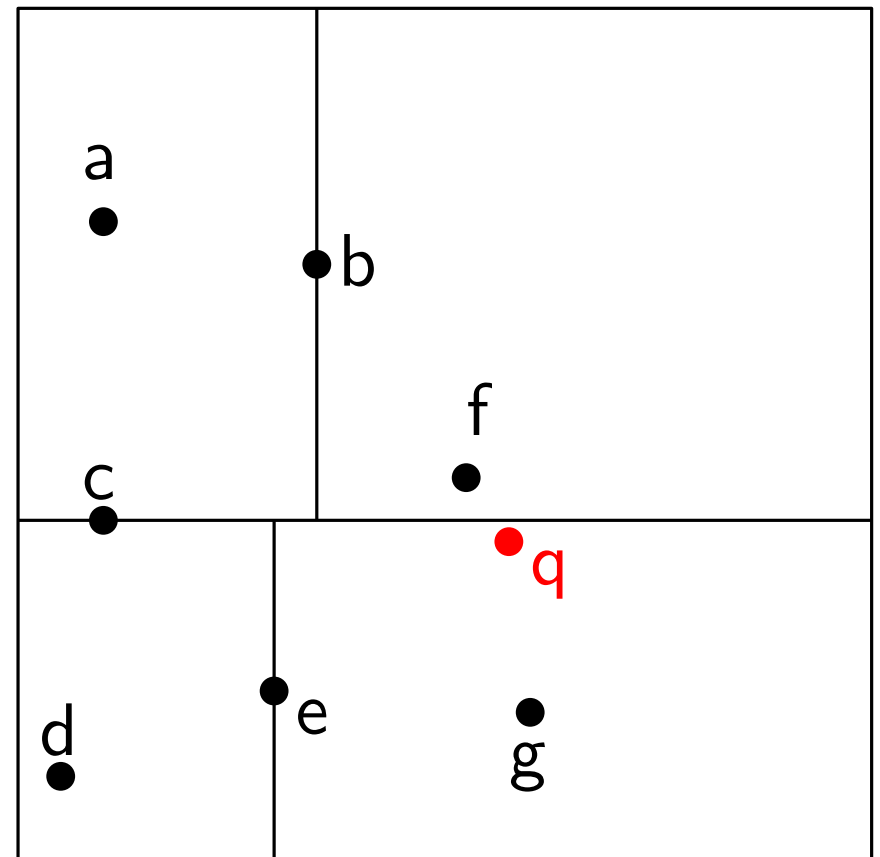
output: kd-tree t' with all points of t and q

```
1 if  $t$  is null then return kd-node( $q$ )
2 pivot  $\leftarrow$   $t$ .point
3 if  $q_{cd} < \text{pivot}_{cd}$  then
4      $t$ .left  $\leftarrow$  kdt-insert ( $t$ .left,  $q$ ,  $(cd+1) \bmod k$ )
5 else
6      $t$ .right  $\leftarrow$  kdt-insert ( $t$ .right,  $q$ ,  $(cd+1) \bmod k$ )
7 end
8 return  $t$ 
```

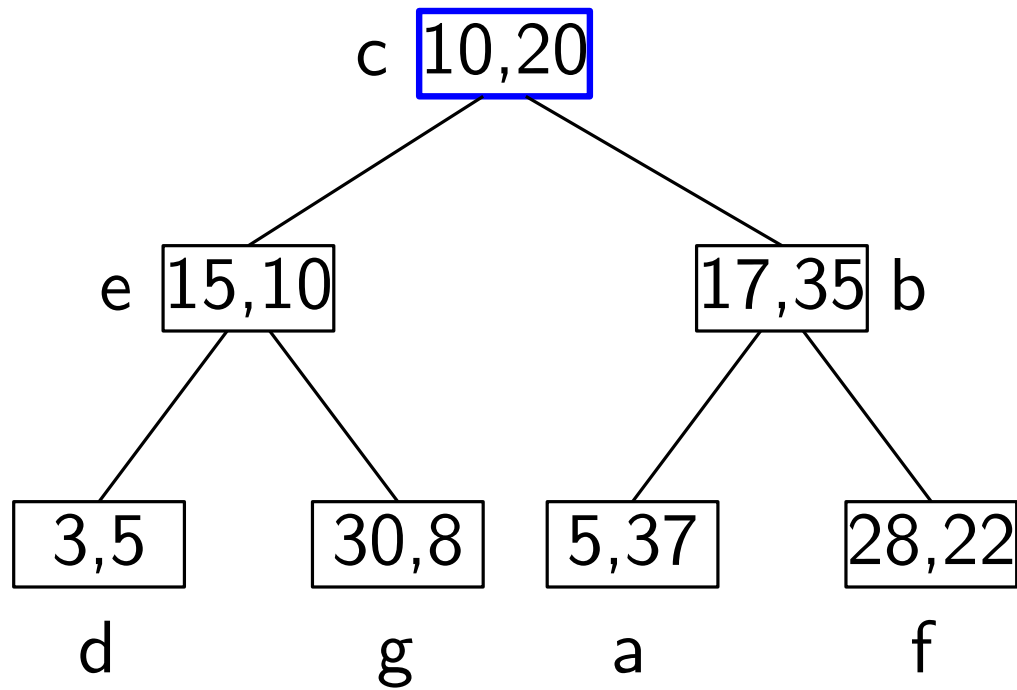
Nearest neighbour search



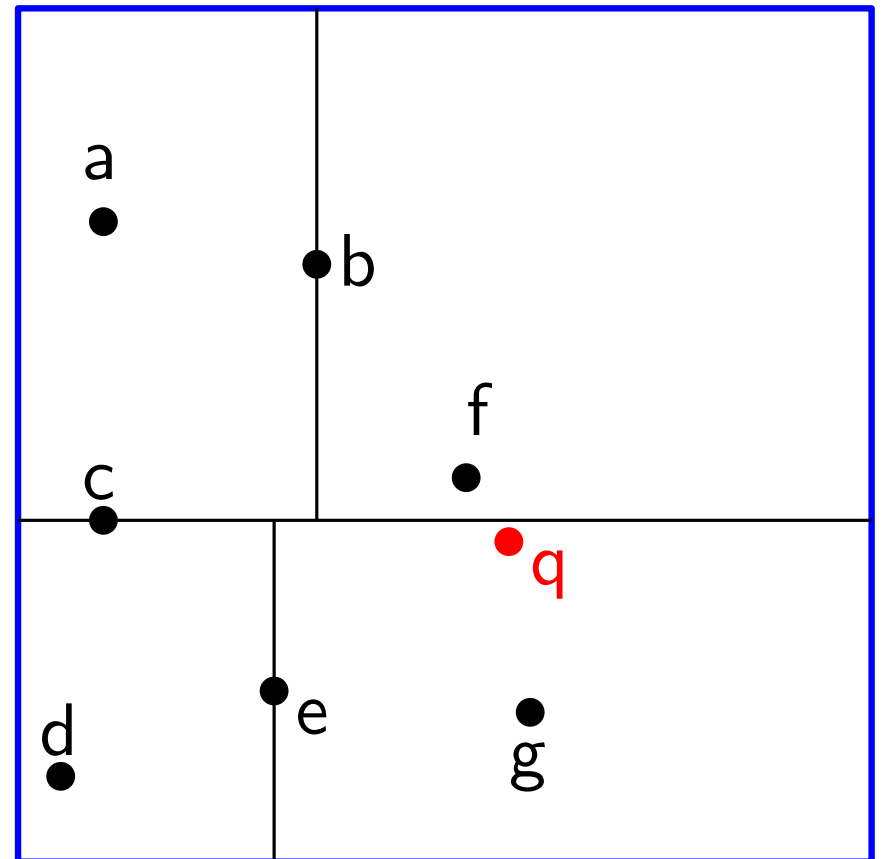
q (29,19)



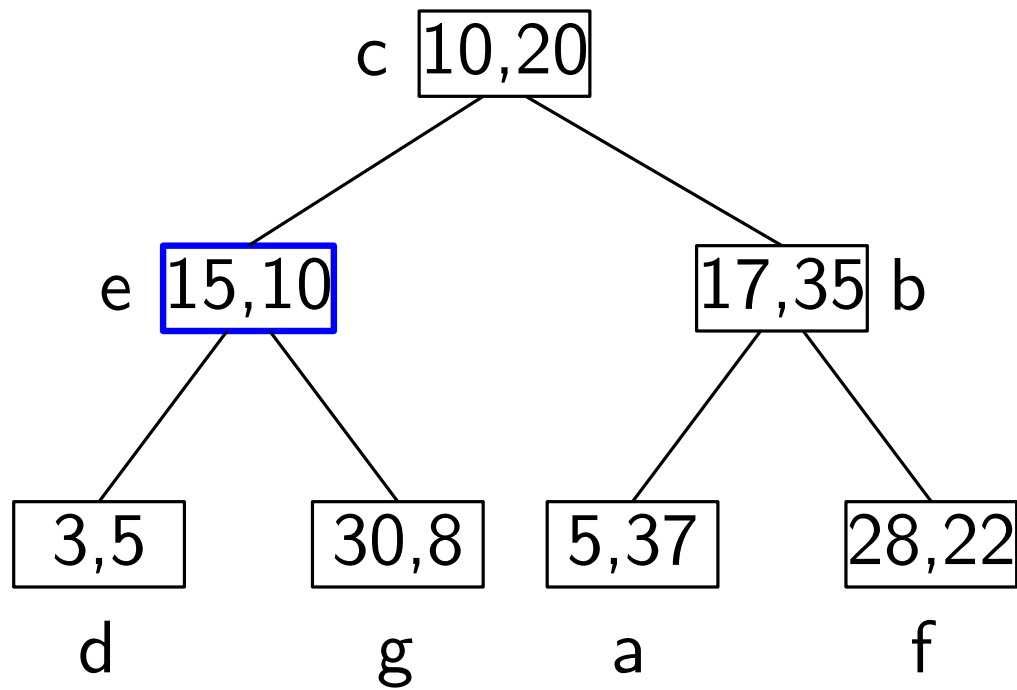
Nearest neighbour search



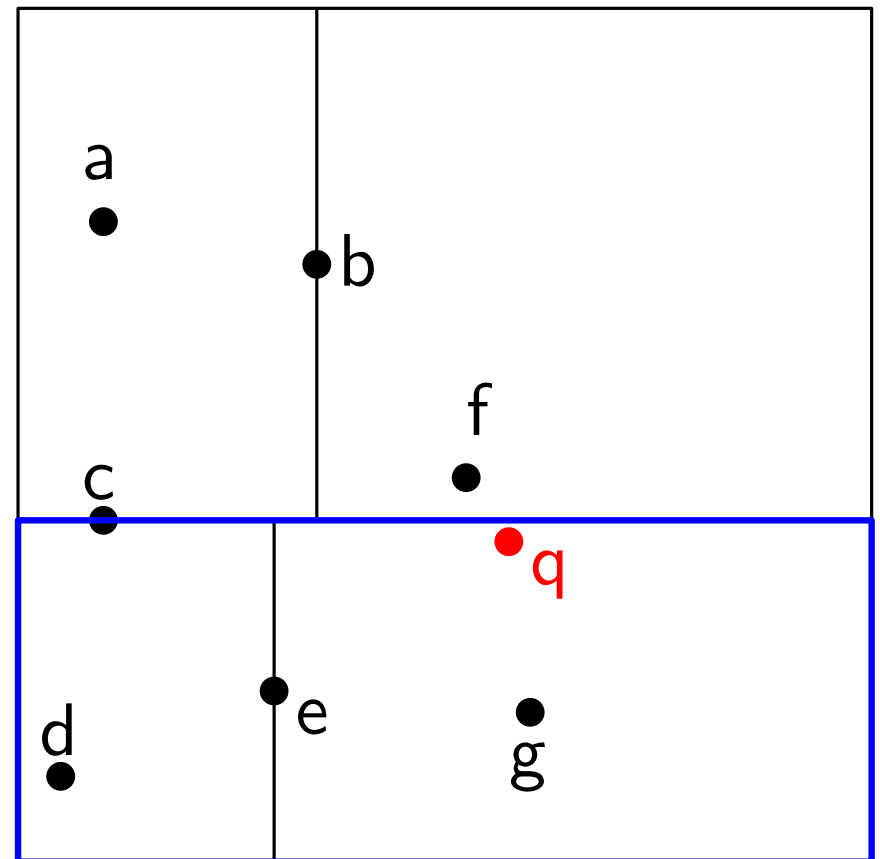
q (29, 19)



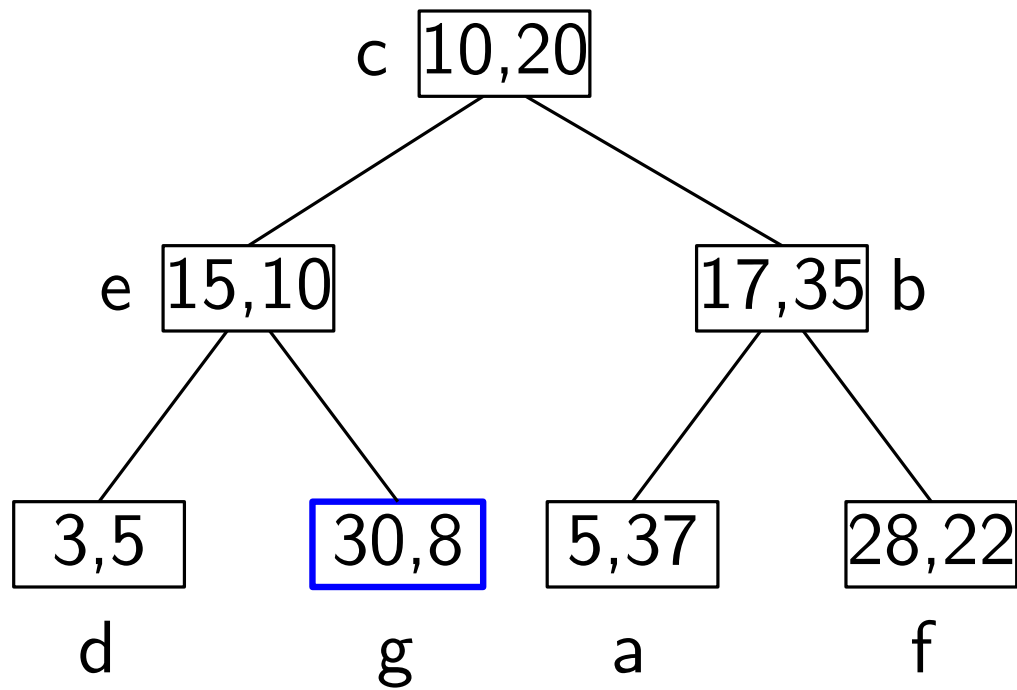
Nearest neighbour search



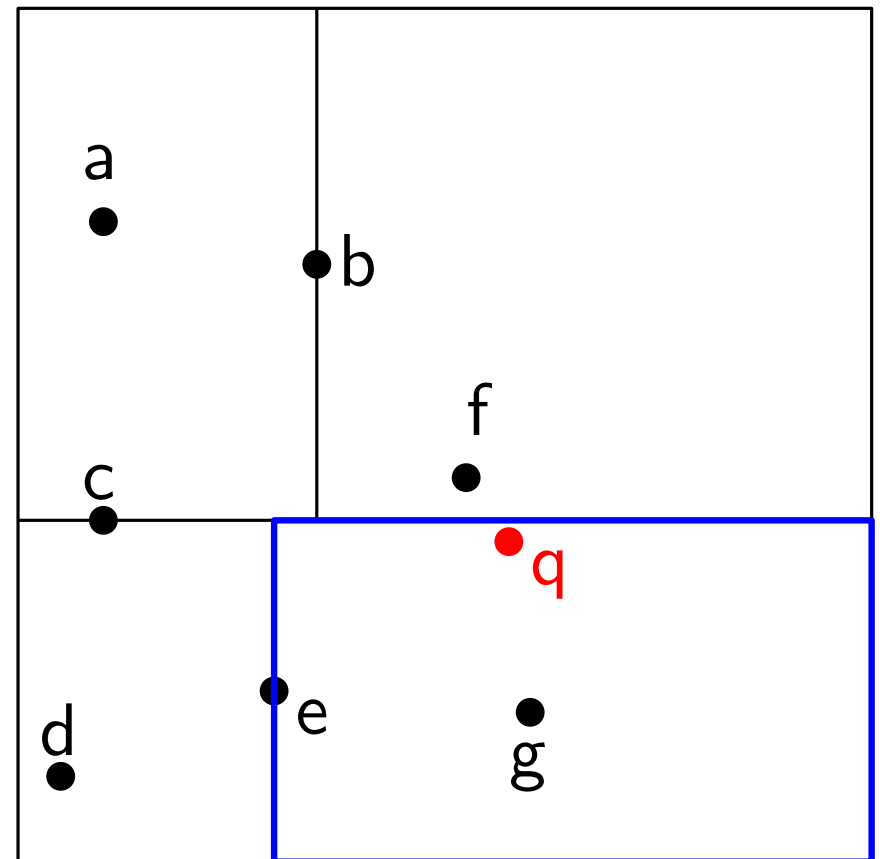
q (29, 19)



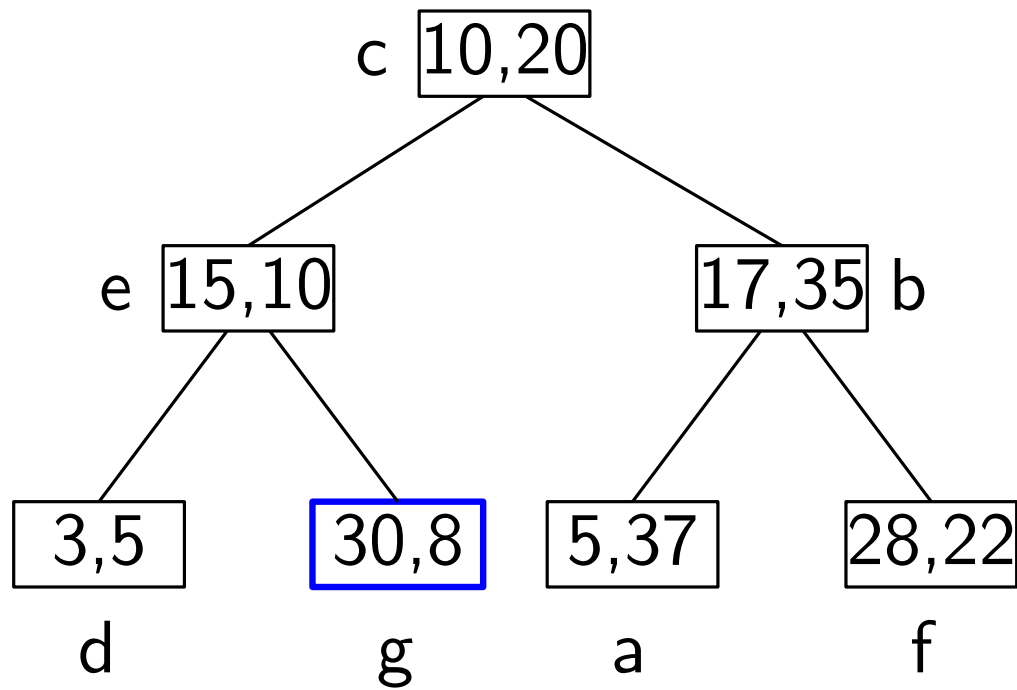
Nearest neighbour search



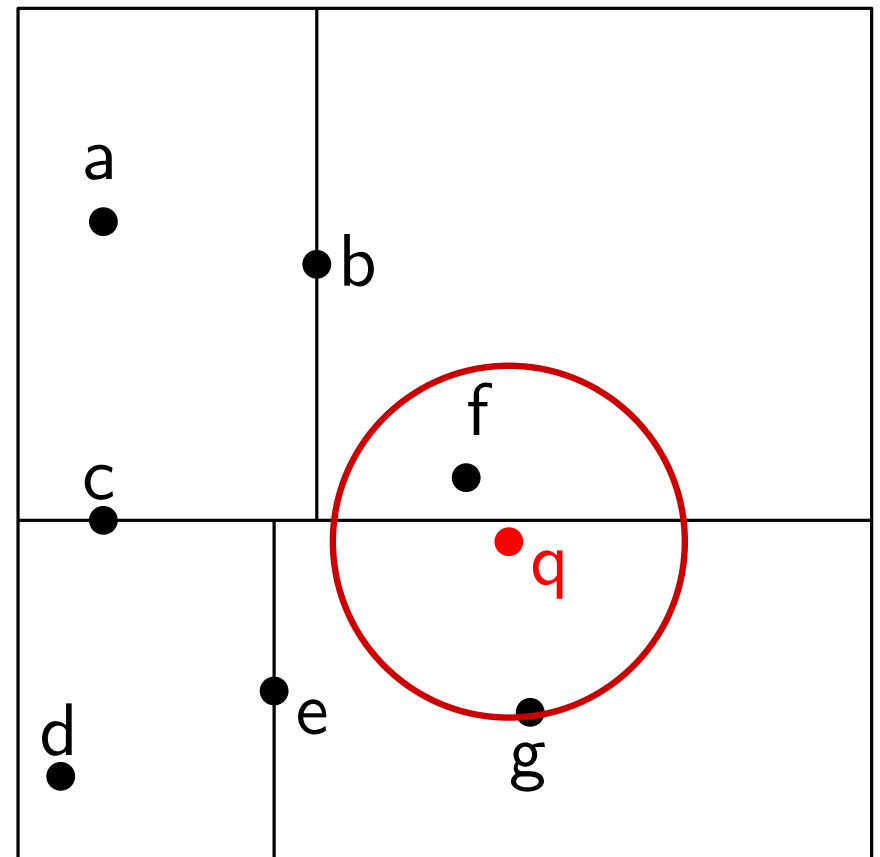
q (29,19)



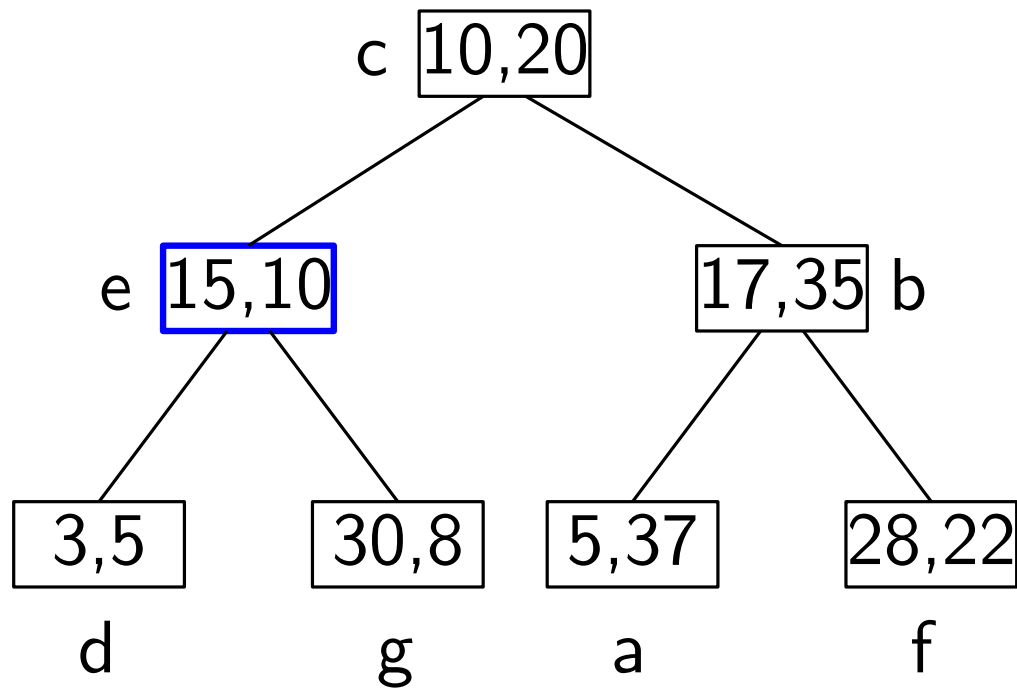
Nearest neighbour search



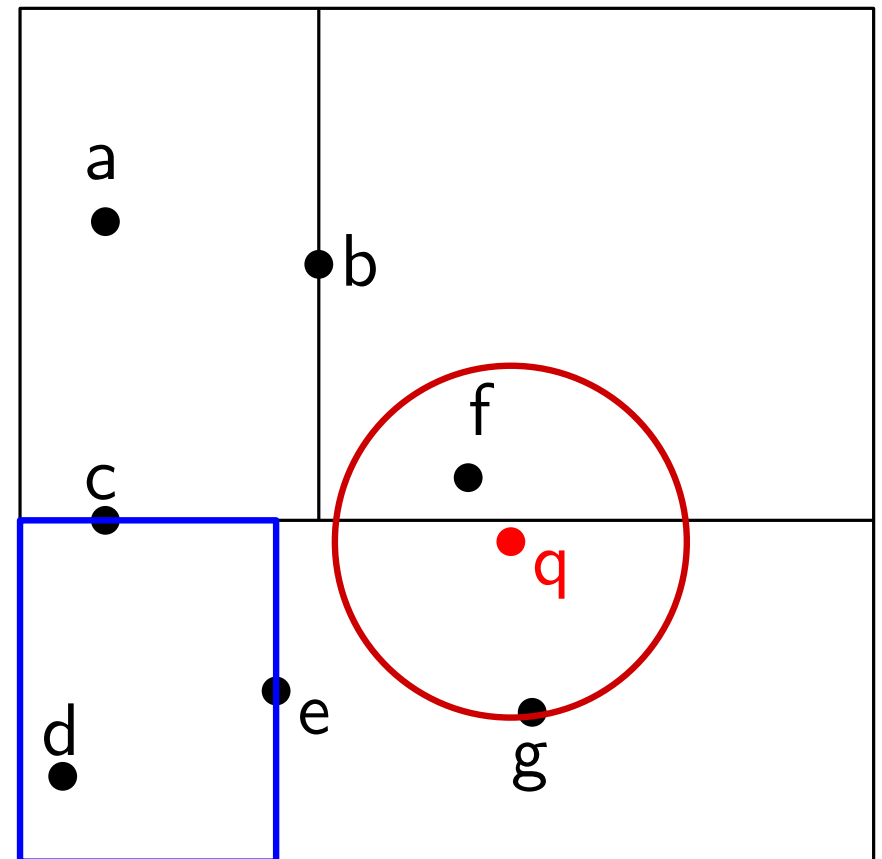
q (29,19)



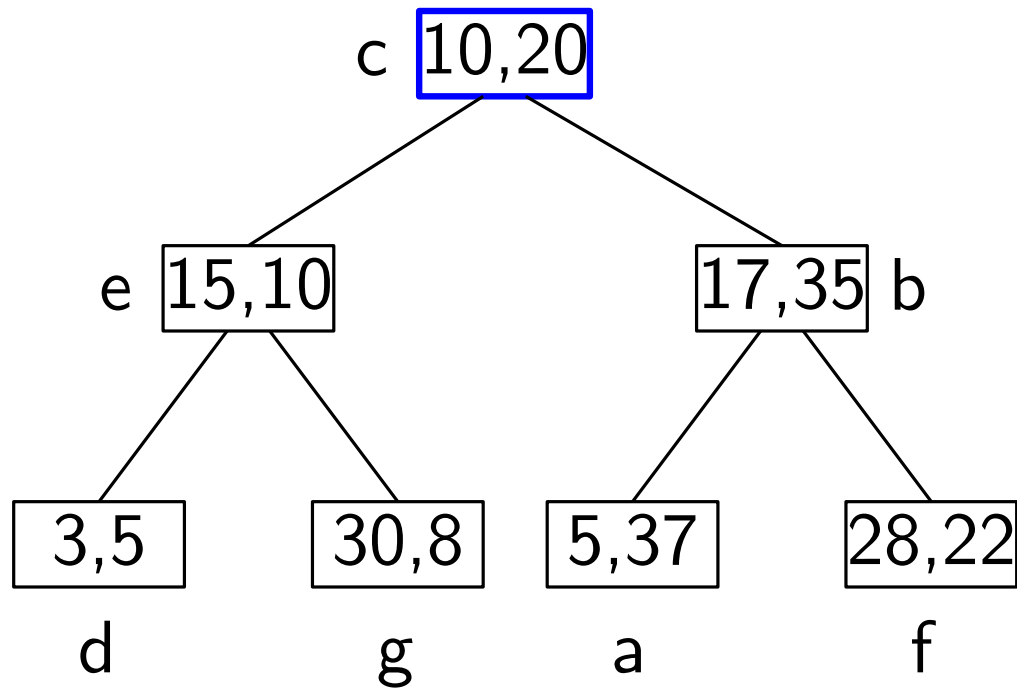
Nearest neighbour search



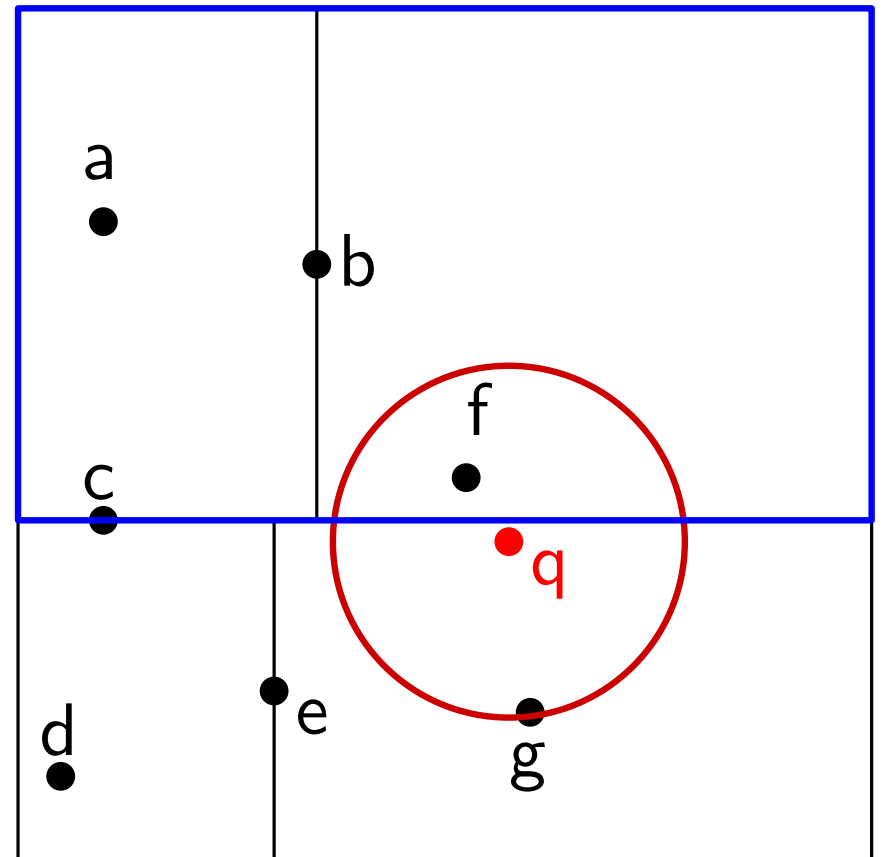
q (29,19)



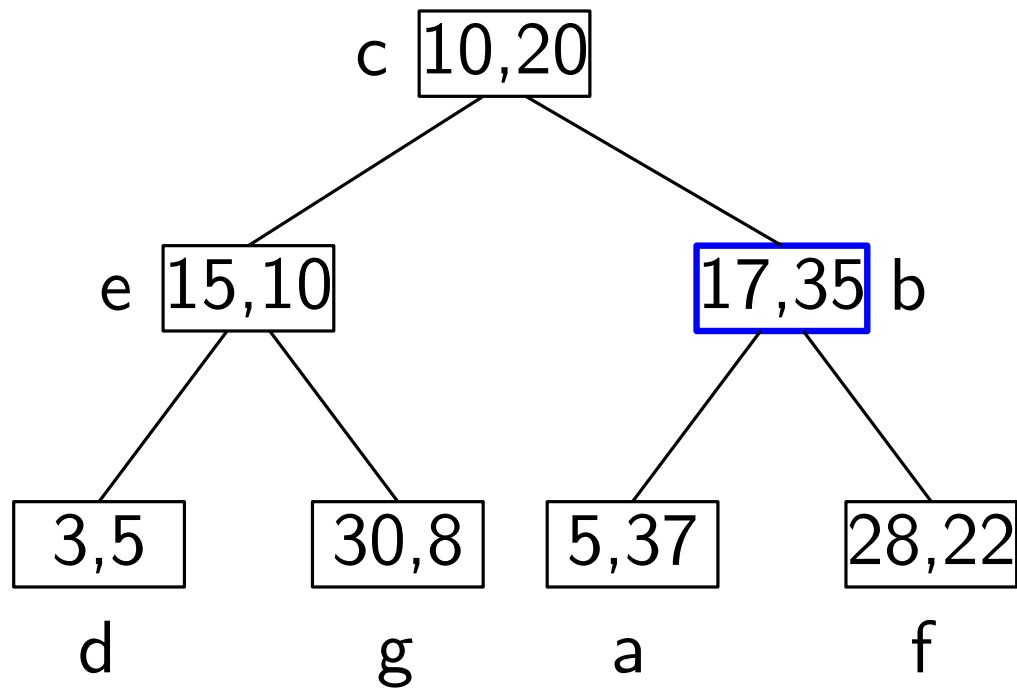
Nearest neighbour search



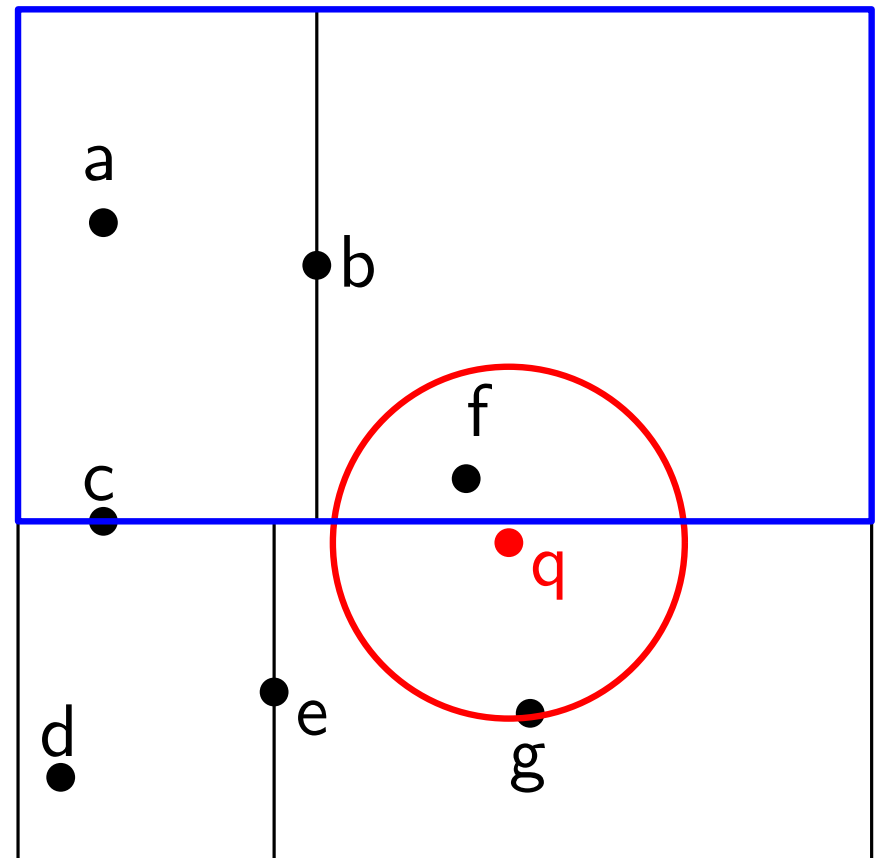
q (29, 19)



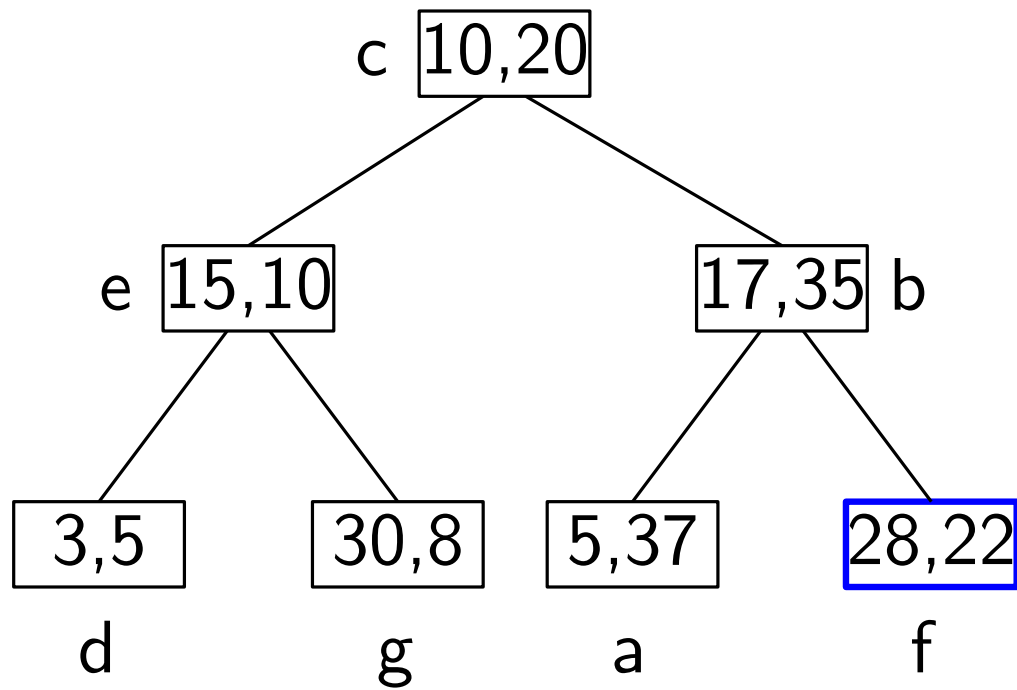
Nearest neighbour search



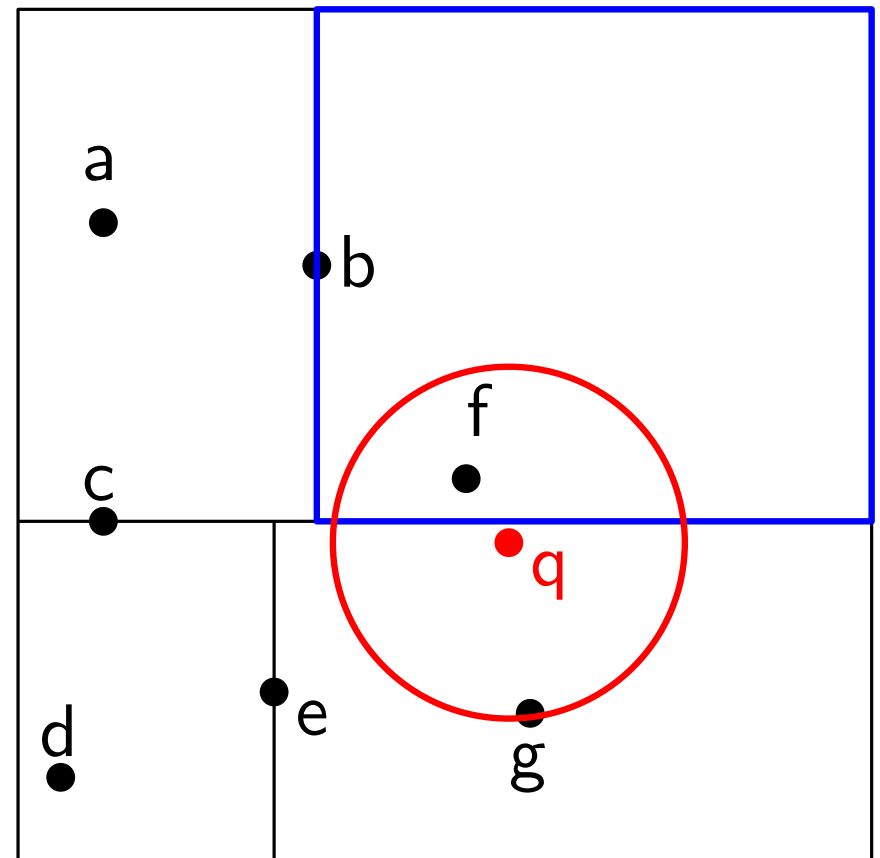
q (29,19)



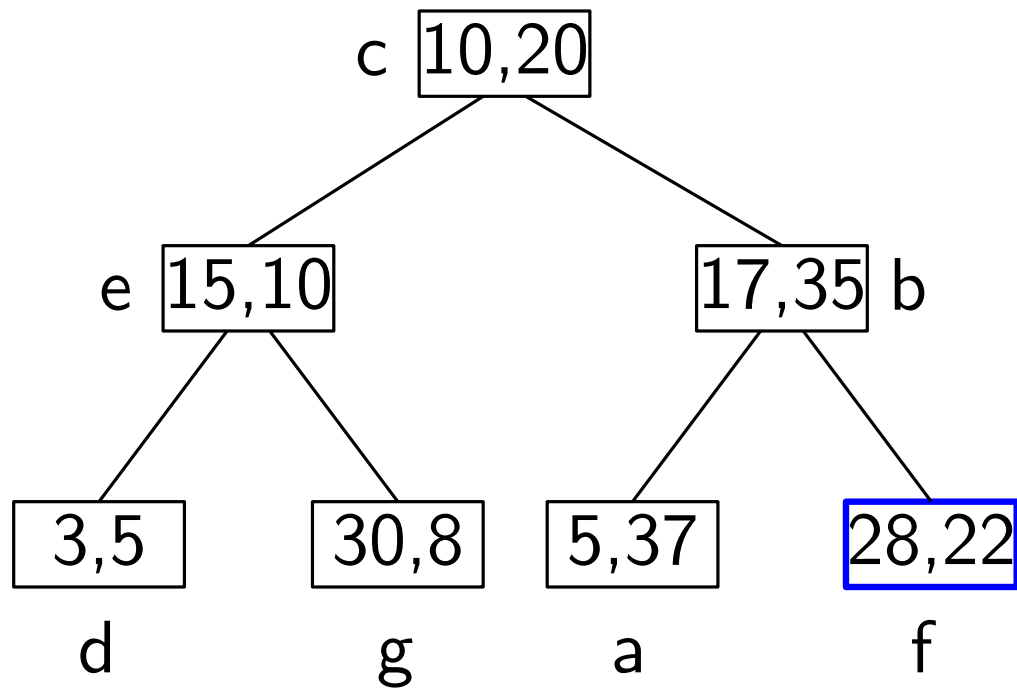
Nearest neighbour search



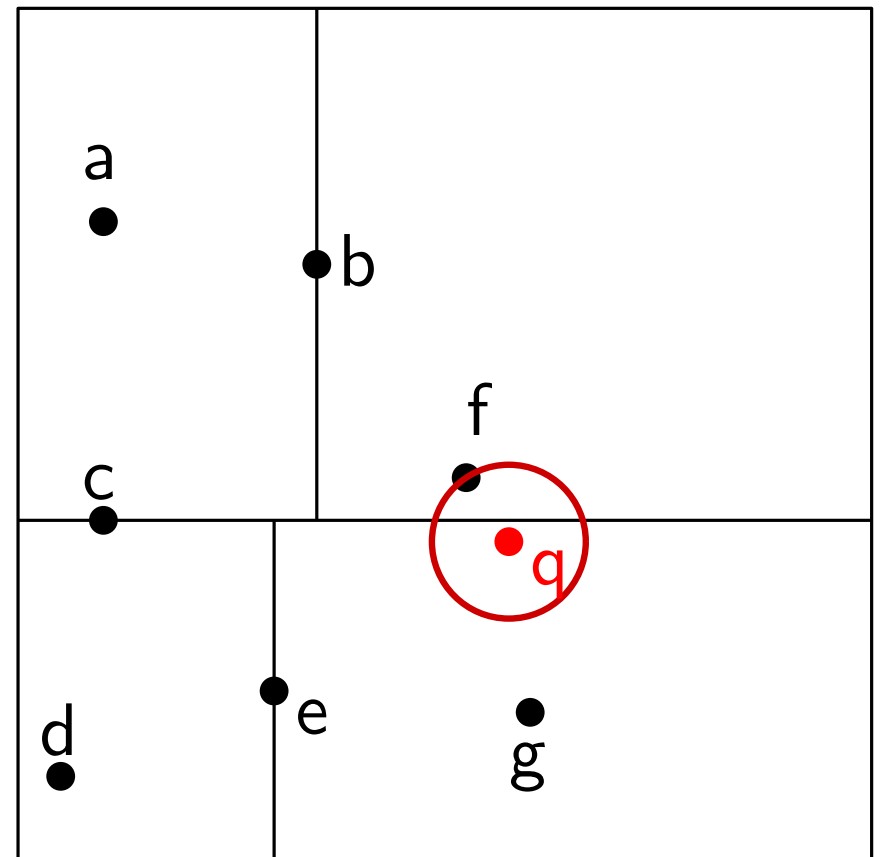
q (29,19)



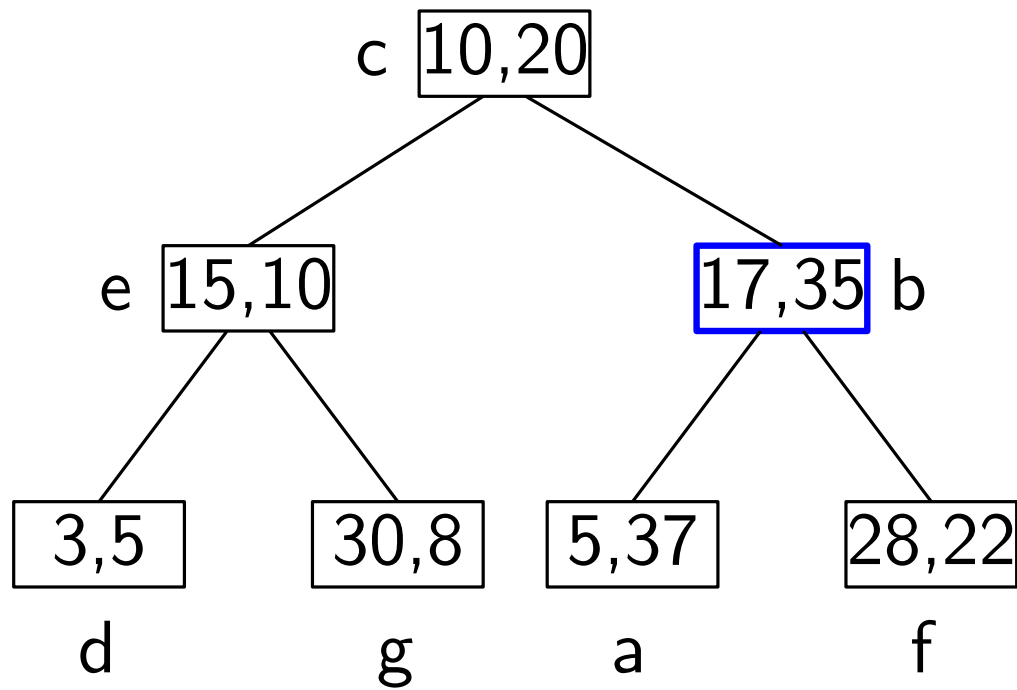
Nearest neighbour search



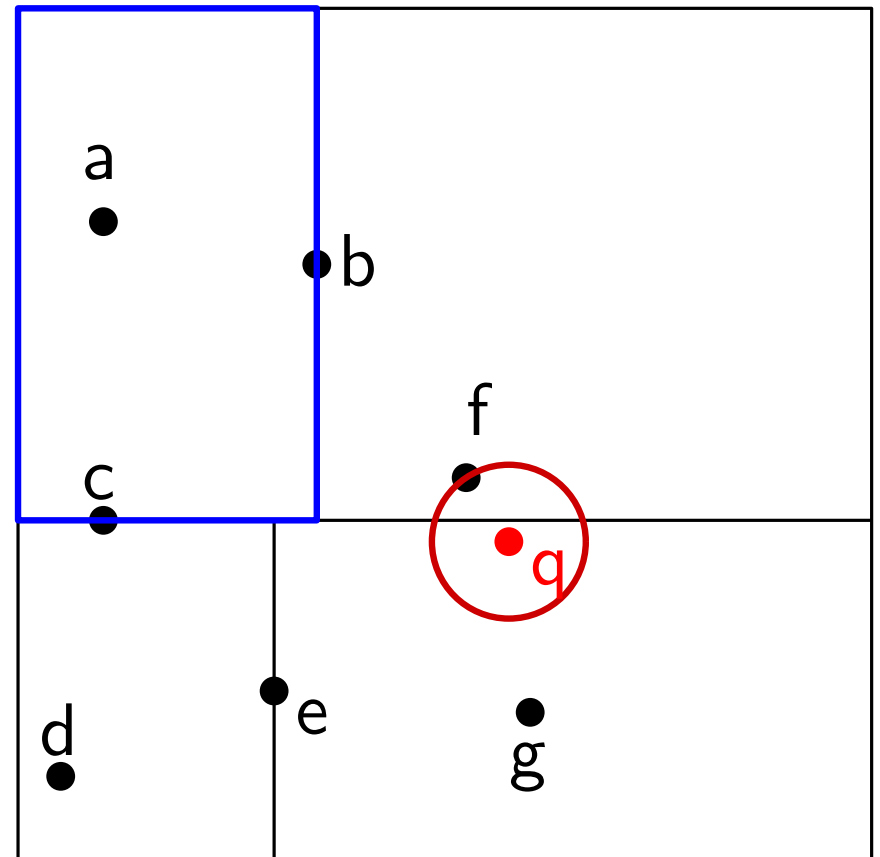
q (29,19)



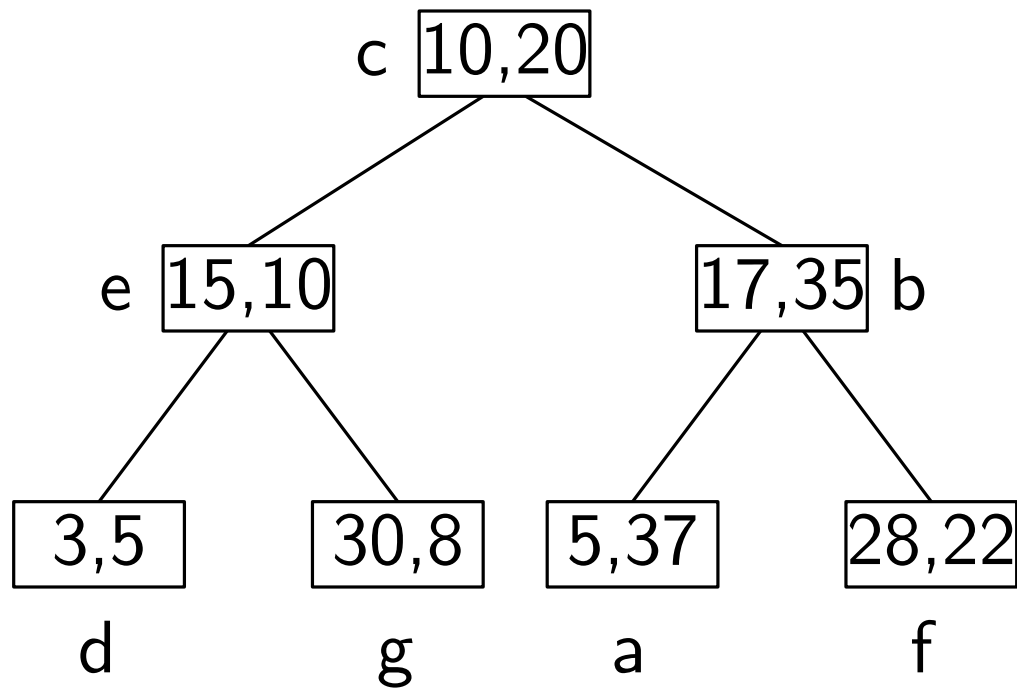
Nearest neighbour search



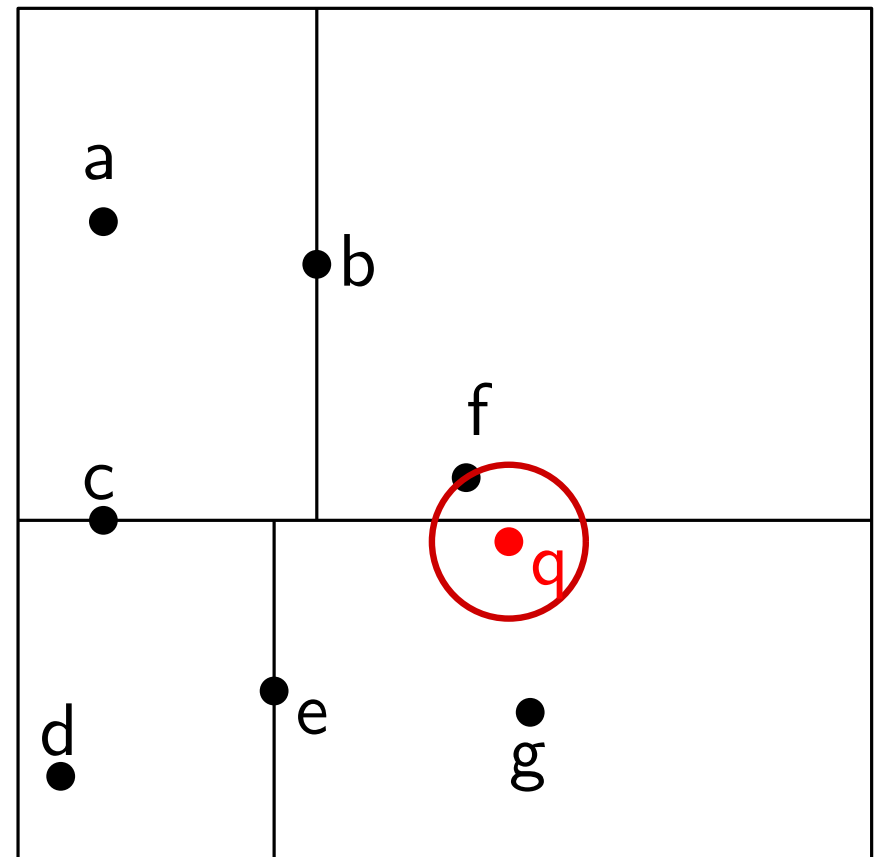
q (29,19)



Nearest neighbour search



q (29,19)



Nearest neighbour search with kd-tree

Algorithm: kdt-NearestNeighbour (t, q, box)

Global variables:

nndist = ∞

nn = -

if t is null **then** return

if $q_{t.cd} < t.point_{t.cd}$ **then**

 kdt-NearestNeighbour (t.left, q, left halfbox)

else

 kdt-NearestNeighbour (t.right, q, right halfbox)

end

if $d(t.point, q) < nndist$ **then**

$nndist \leftarrow d(t.point, q)$

$nn \leftarrow t.point$

end

if other halfbox intersects circle (centre=q, radius=nndist) **then**

 kdt-NearestNeighbour (other subtree, q, other halfbox)

end

End of Lecture 13

Nearest neighbour search

Algorithm: kdt-NearestNeighbour (t, q, box)

input : kd-tree t, query point q, bounding box box

output: Nearest neighbour nn

```
1  if t is null then return
2  pivot, cd  $\leftarrow$  t.point, t.cd
3  cut box into left-halfbox and right-halfbox using pivot and cd
4  if  $q_{cd} < \text{pivot}_{cd}$  then
5      nearer-subtree, nearer-halfbox  $\leftarrow$  t.left, left-halfbox
6      farther-subtree, farther-halfbox  $\leftarrow$  t.right, right-halfbox
7  else
8      nearer-subtree, nearer-halfbox  $\leftarrow$  t.right, right-halfbox
9      farther-subtree, farther-halfbox  $\leftarrow$  t.left, left-halfbox
10 end
11 nn, dist  $\leftarrow$  kdt-NearestNeighbour (nearer-subtree, q, nearer-halfbox)
12 nndist  $\leftarrow$  min (dist, nndist)
13 if farther-halfbox overlaps circle with radius nndist then
14     nn', dist'  $\leftarrow$  kdt-NearestNeighbour (farther-subtree, q, farther-halfbox)
15 end
16 return t
```